
Algoritmo de Maximización de Expectativas (EM) para Modelos de Mezclas Gaussianas (GMM)

1. Introducción Teórica

El algoritmo **EM (Expectation-Maximization)** es un método iterativo diseñado para encontrar estimaciones de máxima verosimilitud de los parámetros en modelos estadísticos que dependen de **variables latentes** (variables ocultas o no observadas).

Un **Modelo de Mezclas Gaussianas (GMM)** asume que todos los puntos de datos del dataset se generan a partir de una mezcla de un número finito K de distribuciones gaussianas con parámetros desconocidos. A diferencia del algoritmo K-Means, que realiza una asignación rígida (*hard clustering*), GMM implementa una asignación probabilística difusa (*soft clustering*), calculando la probabilidad de que cada punto pertenezca a cada clúster.

Cada una de las K componentes gaussianas se describe matemáticamente mediante la función de densidad de probabilidad multivariada:

$$N(x | \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{D/2} |\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)\right)$$

Donde:

- x : Vector de características del punto de datos.
- μ_k : Vector de medias de la componente k .
- Σ_k : Matriz de covarianza de la componente k .
- D : Número de dimensiones o características del dataset.

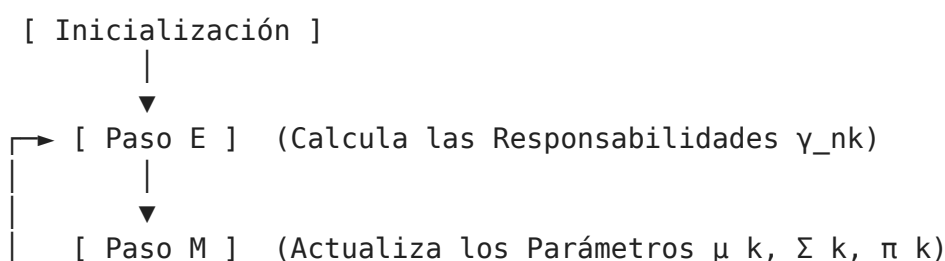
La densidad de probabilidad de la mezcla completa está dada por:

$$p(x) = \sum_{k=1}^K \pi_k N(x | \mu_k, \Sigma_k)$$

Donde π_k representa el peso de mezcla de la componente k , cumpliendo estrictamente con $\sum_{k=1}^K \pi_k = 1$ y $\pi_k \geq 0$.

2. El Ciclo de Vida del Algoritmo EM (Paso a Paso)

El algoritmo opera alternando de forma continua entre dos pasos fundamentales hasta lograr la convergencia:



Paso 0: Inicialización de Parámetros

Antes de iterar, se deben definir valores iniciales para las medias μ_k , las covarianzas Σ_k y los pesos π_k . Nuestro código implementa dos estrategias:

1. **random_from_data** : Elige K puntos aleatorios del dataset como medias y define covarianzas como pequeñas matrices identidad escaladas ($\Sigma_k = I \times 0.01$).
2. **random** : Genera centros flotantes puramente aleatorios distribuidos uniformemente en el hipercubo del espacio de características $[0, 1]^D$.

Paso 1: Expectativa (Paso E)

Calcula las **responsabilidades** (γ_{nk}), que representan la probabilidad condicional de que el componente k haya generado el dato x_n , dadas las estimaciones de los parámetros actuales:

$$\gamma_{nk} = p(z_k = 1 \mid x_n) = \frac{\pi_k N(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j N(x_n \mid \mu_j, \Sigma_j)}$$

Paso 2: Maximización (Paso M)

Utiliza las responsabilidades calculadas en el Paso E para actualizar los parámetros de los componentes de forma óptima mediante estimación por máxima verosimilitud ponderada:

1. **Número efectivo de puntos asignados al componente k :**

$$N_k = \sum_{n=1}^N \gamma_{nk}$$

2. **Nuevas Medias (μ_k):**

$$\mu_k^{\text{nuevo}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} x_n$$

3. **Nuevas Matrices de Covarianza (Σ_k):**

$$\Sigma_k^{\text{nuevo}} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (x_n - \mu_k^{\text{nuevo}})(x_n - \mu_k^{\text{nuevo}})^T$$

4. **Nuevos Pesos de Mezcla (π_k):**

$$\pi_k^{\text{nuevo}} = \frac{N_k}{N}$$

Criterio de Convergencia

El algoritmo evalúa la **Log-Verosimilitud total (Lower Bound)** en cada paso:

$$\ln p(X \mid \mu, \Sigma, \pi) = \sum_{n=1}^N \ln \left(\sum_{k=1}^K \pi_k N(x_n \mid \mu_k, \Sigma_k) \right)$$

El bucle se detiene de forma controlada cuando el cambio promedio por cada muestra matemática entre iteraciones cae por debajo de un umbral preestablecido (`tol`):

$$\frac{|\ln p(X)^{\text{actual}} - \ln p(X)^{\text{anterior}}|}{N} < \text{tol}$$

3. Análisis Detallado del Código e Implementación con Ejemplos

A continuación, descompondremos cada función del script desarrollado en Perl utilizando la biblioteca de tensores `AI::MXNet`.

3.1 Inicialización de Parámetros (`initialize_parameters`)

Esta subrutina se encarga de romper la simetría inicial del espacio matemático antes de entrar al bucle.

```
In [1]: use strict;
use warnings;
use Data::Dump qw(dump);
use sml qw(show_plot);
use AI::MXNet qw(mx nd);
IPerl->load_plugin('Chart::Plotly');
```

```
In [2]: sub initialize_parameters {
    my ($self, $X, $K, $method) = @_;
    my ($N, $D) = @{$X->shape};

    $method //= 'random_from_data';

    if ($method eq 'random_from_data') {
        # --- MÉTODO 1: RANDOM FROM DATA (Estilo Correcto Sklearn) ---
        # 1. Seleccionar K centros reales del dataset de forma aleatoria
        my $indices = nd->random_choice($N, $K, replace=>0);
        my $means = nd->take($X, $indices, axis => 0);

        # 2. Escalar la matriz identidad para reflejar la varianza real del espacio [0, 1]
        # Multiplicar por 0.01 o 0.005 hace que las campanas iniciales capten vecindarios
        my @covs = map { nd->eye($D) * 0.01 } 1 .. $K;

        # 3. Pesos iniciales equitativos
        my $weights = nd->ones([$K]) / $K;

        # 4. Forzar un E-step inicial para propagar responsabilidades realistas
        my ($resp, $lower) = sml->e_step($X, $means, \@covs, $weights);
        return $resp;
    }
    elsif ($method eq 'random') {
        # --- MÉTODO 2: RANDOM CORREGIDO (Muestreo uniforme en el espacio [0,1]) ---
        # En lugar de rellenar la matriz de responsabilidades con ruido uniforme
        # (lo que promedia todo al centro), generamos K centros aleatorios flotantes
        # dentro del rango real de nuestros datos normalizados [0, 1].
        my $means = nd->random->uniform(low => 0.0, high => 1.0, shape => [$K, $D]);

        # Inicializamos con covarianzas pequeñas y locales para que capten vecindarios
        my @covs = map { nd->eye($D) * 0.01 } 1 .. $K;

        # Pesos iniciales equitativos
        my $weights = nd->ones([$K]) / $K;

        # Propagamos mediante el E-step para generar responsabilidades nítidas y reales
```

```

    my ($resp, $lower) = sml->e_step($X, $means, \@covs, $weights);
    return $resp;
  } else {
    die "Método de inicialización desconocido: $method";
  }
}
sml->add_to_class('initialize_parameters', \&initialize_parameters);

```

Out[2]: *sml::initialize_parameters

- **Detalle Clave:** En lugar de retornar matrices difusas planas (ruido uniforme), forzamos un `e_step` inicial usando campanas gaussianas angostas (`nd->eye($D) * 0.01`). Esto permite crear vecindarios selectivos locales y evita que todas las medias colapsen hacia el centro en la primera iteración.

3.2 Densidad Gaussiana Estable (`estimate_log_gaussian_prob`)

Calcular la probabilidad gaussiana multivariada directa puede provocar desbordamiento numérico o subdesbordamiento (*underflow*). Esta función utiliza la **Descomposición de Cholesky** ($\Sigma = LL^T$) y el operador de sustitución triangular eficiente `trsm` para mantener una precisión numérica óptima.

```

In [3]: sub estimate_log_gaussian_prob {
  my ($self, $X, $mean, $cov) = @_;
  my ($N, $D) = @{$X->shape};

  # 1. Añadir regularización a la diagonal idéntica a reg_covar de sklearn
  my $cov_stable = $cov + (nd->eye($D) * 1e-6);

  # 2. Descomposición de Cholesky estable: cov = L * L^T
  # MXNet devuelve una matriz triangular inferior L
  my $L = nd->linalg->potrf($cov_stable);

  # 3. Calcular Log-Determinante de forma segura usando la diagonal de L
  # log(det(Sigma)) = 2 * sum(log(diag(L)))
  # Extraemos la diagonal usando slicing dinámico indexado
  my @diag_elements;
  for my $i (0 .. $D - 1) {
    push @diag_elements, nd->log($L->slice($i, $i));
  }
  my $log_det = nd->add_n(@diag_elements) * 2.0;

  # 4. Resolver el término cuadrático de Mahalanobis usando linalg->trsm (Sustitución
  # Buscamos resolver de manera estable: L * Y = (X - mean)^T
  my $X_centered = nd->broadcast_sub($X, $mean)->T;

  # trsm resuelve sistemas triangulares de manera ultra-eficiente
  my $Y = nd->linalg->trsm($L, $X_centered, alpha => 1.0, rightside => 0, lower => 1,

  # El término cuadrático equivale a la suma de los cuadrados de Y por columnas
  my $squad = nd->sum(nd->square($Y), axis => 0);

  # 5. Combinar todos los factores en la constante Gaussiana multivariada
  my $pi = 3.141592653589793;
  my $log_prob = -0.5 * ( $D * log(2 * $pi) + $log_det + $squad );

  return $log_prob;
}
sml->add_to_class('estimate_log_gaussian_prob', \&estimate_log_gaussian_prob);

```

Out[3]: *sml::estimate_log_gaussian_prob



Ejemplo Numérico de Juguete para Estudiantes

Imaginemos que evaluamos un único punto bidimensional ($D = 2$) en un componente centrado en el origen:

- Dato: $x = [0.5, 0.5]$
- Media: $\mu = [0, 0]$
- Covarianza: $\Sigma = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$

Paso a Paso Matemático:

1. **Cholesky:** $L = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$ ya que $LL^T = \Sigma$.

2. **Log-Determinante:** $\ln |\Sigma| = 2 \times (\ln(0.5) + \ln(0.5)) = 2 \times (-0.6931 - 0.6931) = -2.7725$

3. **Término Cuadrático (Sustitución):**

$$LY = (x - \mu)^T \implies \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \implies Y = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

4. **Distancia Cuadrática:** $\text{quad} = 1^2 + 1^2 = 2$.

5. **Log-Probabilidad:**

$$\ln N = -0.5 \times (2 \times \ln(2\pi) + (-2.7725) + 2) = -0.5 \times (3.6757 - 2.7725 + 2) = -1.4516$$

3.3 Paso de Expectativa (e_step)

Agrupar las probabilidades gaussianas ponderadas de todas las muestras y aplica el truco matemático **Log-Sum-Exp** para normalizar de forma numéricamente segura las responsabilidades.

```
In [4]: sub e_step {
  my ($self, $X, $means, $covs, $weights) = @_;
  my $K = $weights->shape->[0];
  my @cols;

  for my $k (0 .. $K - 1) {
    my $slice = $means->slice($k);
    my $log_prob = sml->estimate_log_gaussian_prob(
      $X,
      $slice,
      $covs->[$k]
    );
    push @cols, $log_prob + log($weights->at($k)->asscalar);
  }

  my $log_resp = nd->stack(@cols, axis => 1);

  # Truco Log-Sum-Exp para estabilidad numérica
  my $max_log = $log_resp->max(axis => 1, keepdims => 1);
  my $log_sum_exp = $max_log + nd->log(
    nd->exp(nd->broadcast_sub($log_resp, $max_log))->sum(axis => 1, keepdims => 1)
  );

  my $responsibilities = nd->exp(nd->broadcast_sub($log_resp, $log_sum_exp));
  my $lower_bound = $log_sum_exp->sum->asscalar;

  return ($responsibilities, $lower_bound);
}
sml->add_to_class('e_step', \&e_step);
```

Out[4]: *sml::e_step

3.4 Paso de Maximización (m_step)

Toma la matriz distributiva `$resp` de dimensiones $[N, K]$ y actualiza in-place las estructuras geométricas físicas mediante operaciones vectorizadas de álgebra lineal elemental.

```
In [5]: sub m_step {
  my ($self, $X, $resp) = @_;
  my ($N, $D) = @{$X->shape};
  my $K = $resp->shape->[1];

  my $Nk = $resp->sum(axis => 0);
  # Ajuste de pesos idéntico a sklearn
  my $weights = $Nk / $N;
  my (@means, @covs);

  for my $k (0 .. $K - 1) {
    my $nk = $Nk->at($k)->asscalar;
    if ($nk < 1e-10) { $nk = 1e-10; }

    my $r = $resp->slice(':', $k)->reshape([-1, 1]);

    # Actualización de Medias robusta con broadcasting explícito
    my $mean = nd->sum(nd->broadcast_mul($X, $r), axis => 0) / $nk;
    push @means, $mean;

    # Actualización de Covarianzas ponderada
    my $diff = nd->broadcast_sub($X, $mean);
    my $weighted_diff = nd->broadcast_mul($diff, $r);

    # Producto matricial clásico para la forma de la covarianza [D, D]
    my $cov = nd->dot($weighted_diff->T, $diff) / $nk;

    push @covs, $cov;
  }

  my $means_stacked = nd->stack(@means);
  return ($means_stacked, \@covs, $weights);
}
sml->add_to_class('m_step', \&m_step);
```

Out[5]: *sml::m_step

3.5 El Bucle de Optimización Global (fit)

La subrutina `fit` actúa como el orquestador o motor principal de ejecución. Su función es iterar de manera controlada coordinando las fases de expectativa y maximización, gestionando la convergencia numérica y controlando el ciclo de parada.

```
In [6]: sub fit {
  my ($self, $X, $K, %args) = @_;
  my $max_iter = $args{max_iter} // 100;
  my $tol = $args{tol} // 1e-3;
  my $init_param = $args{init_params} // 'random_from_data'; # Recibe el parámetro

  # Inicializamos obteniendo la matriz de responsabilidades
  my $resp = sml->initialize_parameters($X, $K, $init_param);

  my $lower_old = -1e100;
```

```

my (@history, @plots);
my ($means, $covs, $weights);
my $N = $X->len;
my $has_converged = 0;

for my $iter (1 .. $max_iter) {

  # 1. Paso M: Estimar parámetros a partir de las responsabilidades vigentes
  ($means, $covs, $weights) = sml->m_step($X, $resp);

  # 2. Paso E: Evaluar las nuevas responsabilidades y el Lower Bound
  my $lower;
  ($resp, $lower) = sml->e_step($X, $means, $covs, $weights);

  # -----
  # printf "Iter %d LL=%.4f\n", $iter, $lower;
  push @history, $lower;

  my $assignments = nd->argmax($resp, axis => 1);

  push @plots, sml->plot_current_state(
    $X, $means, $covs, $assignments, $iter, $args{header}, $K
  ) if $args{plot_steps};

  my $diff = abs($lower - $lower_old) / $N; # <--- Dividir para N

  if ($diff < $tol) {
    $has_converged = 1;
    print "Convergence achieved in the iteration $iter.\n";
    last;
  }

  $lower_old = $lower;
}

if (not $has_converged) {
  warn "\nConvergence warning: Initialization $max_iter did not converge. "
    . "Try different init parameters, or increase max_iter, tol "
    . "or check for degenerate data.\n\n";
}

return ($resp, $means, $covs, \@history, \@plots);
}
sml->add_to_class('fit', \&fit);

```

Out[6]: *sml::fit



Explicación paso a paso para estudiantes:

- Inicialización Léxica:** Se establece el valor `$lower_old` en un número extremadamente negativo (`-1e100`). Esto garantiza que en la primera iteración el cálculo de la diferencia numérica no rompa el programa y permita la entrada al bucle de manera limpia.
- Coordinación EM Inversa:** Noten que el bucle ejecuta primero el `m_step` y luego el `e_step` . Esto se debe a que la función `initialize_parameters` ya devuelve una matriz inicial de responsabilidades `$resp` . El algoritmo aprovecha este estado para estimar inmediatamente las primeras formas geométricas reales.
- Control de Tolerancia Relativa:** La condición de convergencia no evalúa el cambio absoluto de la verosimilitud, sino la ganancia marginal *por cada muestra* (N). Esto previene que datasets muy extensos continúen iterando indefinidamente debido a la acumulación matemática de decimales en tensores de alta dimensionalidad.

3.6 Monitoreo Gráfico de los Componentes (plot_current_state)

Uno de los mayores retos al enseñar Modelos de Mezclas Gaussianas es visualizar cómo las campanas se estiran y se desplazan en tiempo real. Esta función utiliza `Chart::Plotly` para generar gráficos interactivos directos en la sesión del navegador o cuaderno.

¿Cómo funciona el número de contornos en Plotly? Al fijar `size` exactamente igual a la diferencia (`end - start`), le estás diciendo algebraicamente a Plotly: Dibuja el primer contorno en la posición `start` (30%). Avanza un paso equivalente a `size`. La nueva posición evaluada es `start + size`, lo cual da exactamente el valor de `end` (60%). Ahí dibuja el segundo contorno. Si intentara buscar un tercer contorno añadiendo otro paso (`end + size`), este superaría el límite máximo definido en `end`, por lo que el algoritmo de Plotly se detiene de inmediato.

```
In [7]: sub plot_current_state {
  my ($self, $X, $means, $covs, $assignments, $iteration, $header, $n_components) = @_

  # 1. Escala de colores para los puntos
  my $color_scale = [
    [0, 'green'],
    [0.5, 'purple'],
    [1, 'orange']
  ];
  my @contour_colors = ('green', 'purple', 'orange');

  # DINAMIZACIÓN: Extraer límites reales del dataset X para ajustar la ventana visual
  my $x_min = $X->slice(':', 0)->min->asscalar;
  my $x_max = $X->slice(':', 0)->max->asscalar;
  my $y_min = $X->slice(':', 1)->min->asscalar;
  my $y_max = $X->slice(':', 1)->max->asscalar;

  # Añadir un margen del 15% para que los puntos periféricos y contornos no se recorten
  my $x_margin = ($x_max - $x_min) * 0.15 || 0.5;
  my $y_margin = ($y_max - $y_min) * 0.15 || 0.5;

  my $plot_x_min = $x_min - $x_margin;
  my $plot_x_max = $x_max + $x_margin;
  my $plot_y_min = $y_min - $y_margin;
  my $plot_y_max = $y_max + $y_margin;

  # 2. Generación Dinámica de la Malla (Grid) basada en los límites del gráfico
  my $grid_steps = 50;
  my @grid_x;
  my @grid_y;
  for my $i (0 .. $grid_steps - 1) {
    my $factor = $i / ($grid_steps - 1);
    push @grid_x, $plot_x_min + $factor * ($plot_x_max - $plot_x_min);
    push @grid_y, $plot_y_min + $factor * ($plot_y_max - $plot_y_min);
  }

  # Construir el tensor de la malla [grid_steps * grid_steps, 2]
  my @grid_points;
  for my $y_val (@grid_y) {
    for my $x_val (@grid_x) {
      push @grid_points, [$x_val, $y_val];
    }
  }
  my $X_grid = nd->array(\@grid_points);

  # 3. Construcción de los trazos de los anillos
```



```

my @contour_traces;
for my $k (0 .. $n_components - 1) {
    my $mean_k = $means->slice($k);
    my $cov_k = $covs->[$k];

    # Evaluar la densidad de probabilidad multivariada pura en toda la malla
    my $z_flat = multivariate_normal->pdf($X_grid, $mean_k, $cov_k);

    # Extraer el pico de densidad máxima del cluster
    my $z_max_val = $z_flat->max->asscalar;

    # Definimos 2 contornos: el anillo exterior (más bajo/grande) y el interno (más a
    my $contour_start = $z_max_val * 0.30; # Primer anillo (30% de la altura)
    my $contour_end = $z_max_val * 0.60; # Segundo anillo (60% de la altura)

    # El tamaño del paso (size) es exactamente la distancia entre ambos niveles
    my $contour_size = $contour_end - $contour_start;

    my $z_matrix = $z_flat->reshape([$grid_steps, $grid_steps])->asarray;

    # 4. Crear la capa de contornos usando el paso lineal forzado
    my $contour_trace = new Chart::Plotly::Trace::Contour(
        x => \@grid_x,
        y => \@grid_y,
        z => $z_matrix,
        showscale => 0,
        contours => {
            coloring => 'none',
            start => $contour_start, # Comienza en el 30%
            end => $contour_end, # Termina exactamente en el 60%
            size => $contour_size # El paso exacto para que solo dibuje esos d
        },
        line => { color => $contour_colors[$k], width => 2.5 },
        name => "Contour Cluster $k"
    );

    push @contour_traces, $contour_trace;
}

# 4. Capa de puntos reales del dataset
my $points_trace = new Chart::Plotly::Trace::Scatter(
    x => $X->slice(':', 0)->asarray,
    y => $X->slice(':', 1)->asarray,
    mode => 'markers',
    marker => {
        color => ($assignments / ($n_components - 1))->asarray,
        colorscale => $color_scale,
        size => 10,
        opacity => 0.7,
        cmin => 0.0,
        cmax => 1.0
    },
    name => 'Puntos'
);

# 5. Capa de Medias (Cruces Rojas)
my $means_trace = new Chart::Plotly::Trace::Scatter(
    x => $means->slice(':', 0)->asarray,
    y => $means->slice(':', 1)->asarray,
    mode => 'markers',
    marker => { symbol => 'cross', color => 'red', size => 22 },
    name => 'Averages (Centers)'
);

# Ajuste adaptativo del Layout
my $layout = {

```

```

    title => { text => "GMM Clustering - Iteration: $iteration (Density Convergen
xaxis => { title => $header->[0] // 'Feature 0', range => [$plot_x_min, $plot_
yaxis => { title => $header->[1] // 'Feature 1', range => [$plot_y_min, $plot_
width  => 950, height => 500
};

return new Chart::Plotly::Plot(
    traces => [@contour_traces, $points_trace, $means_trace],
    layout => $layout
);
}
sml->add_to_class('plot_current_state', \&plot_current_state);

```

Out[7]: *sml::plot_current_state



Mecánica de Mapeo Numérico-Visual:

- **Normalización de Colores:** Plotly requiere que los valores del mapa de colores estén normalizados en el rango $[0, 1]$. El código transforma dinámicamente las etiquetas de los clústeres dividiéndolas mediante operaciones vectorizadas: $(\$assignments / (\$n_components - 1))$. Para $K = 3$, las etiquetas `0`, `1`, `2` se transforman con precisión en los valores flotantes `0.0`, `0.5`, `1.0`, mapeándose de forma fija a los colores verde, púrpura y naranja respectivamente.
- **Superposición de Capas:** El objeto `Chart::Plotly::Plot` recibe un arreglo ordenado de trazos (`traces => [$points_trace, $means_trace]`). Al colocar `$means_trace` al final, garantizamos que las cruces rojas bidimensionales se rendericen en la capa superior (*Z-index*), evitando que queden ocultas por la densidad de los datos del dataset Iris.

3.6 Carga y Preparación de Datos (Pipeline)

Antes de alimentar un modelo probabilístico sensible a las distancias geométricas como GMM, los datos deben pasar por un proceso riguroso de carga, filtrado dimensional y normalización escalar.

```

In [8]: my ($dataset, $header) = sml->load_csv('/home/jehosha/Downloads/data/iris.csv');
my ($lookup, $rlookup) = sml->str_column_to_int($dataset, -1);
$dataset = nd->array($dataset);

# Extraer columnas 0 y 2 para trabajar en un entorno 2D real
my $X = nd->take($dataset, nd->array([0, 2]), axis => 1);
my $y = $dataset->slice(':', -1);

# Crear un header temporal adaptado a las dos columnas seleccionadas
my $header_2d = [ $header->[0] // 'Sepal Length', $header->[2] // 'Petal Length' ];

# Normalización Min-Max
my $X_normalized = $X;
my $minmax = sml->dataset_minmax($X_normalized);
sml->normalize_dataset($X_normalized, $minmax);

```

Out[8]: <AI::MXNet::NDArray 150x2 @cpu(0)>



Puntos Críticos:

1. **Reducción del Espacio de Características (2D Real):** Seleccionamos exclusivamente las columnas `0` (*Sepal Length*) y `2` (*Petal Length*). Trabajar en dos dimensiones nos permite mapear las transformaciones algebraicas complejas directamente en coordenadas cartesianas (X, Y) legibles en una pantalla sin perder interpretabilidad matemática.

2. **Aislamiento de la Variable Latente Real (\$y)**: Cortamos la última columna del archivo (-1) correspondiente a las especies de Iris. GMM es un algoritmo **no supervisado**, por lo que esta variable se reserva únicamente al final para contrastar el rendimiento del agrupamiento (*clustering*) versus la realidad biológica.
3. **Normalización Min-Max Escalada**: Transforma cada característica mapeándola estrictamente al intervalo flotante [0, 1] mediante la ecuación:

$$x_{\text{norm}} = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}}$$

- **¿Por qué es indispensable para GMM?** Si una característica tuviera un rango de 0 a 1000 y otra de 0 a 1, la primera dominaría por completo el cálculo de la distancia de Mahalanobis en la función de densidad gaussiana, deformando las matrices de covarianza de manera artificial.

```
In [9]: # =====
# EJECUCIÓN DEL ALGORITMO (Configuración para Convergencia Real)
# =====

# Fijar semilla aleatoria global para asegurar reproducibilidad
mx->random->seed(2);
my $K = 3;

# Ajuste iterativo del Modelo de Mezclas Gaussianas
my ($responsibilities, $gmm_means, $gmm_covs, $log_likeliheids, $plots) =
  sml->fit($X_normalized, $K,
          max_iter => 50,          # <--- PERMITIR ITERACIONES
          tol      => 1e-4,        # <--- TOLERANCIA ESTÁNDAR
          header   => $header_2d,
          plot_steps => 0,
          init_params => 'random'); # 'random_from_data'
```

Convergence achieved in the iteration 32.

```
Out[9]: <AI::MXNet::NDArray 150x3 @cpu(0)><AI::MXNet::NDArray 3x2 @cpu(0)>ARRAY(0x55b74671671
0)ARRAY(0x55b7466f2b68)ARRAY(0x55b7466f2b98)
```

3.7 Control de Ejecución, Métricas de Convergencia y Asignación Dura

En este bloque final se inicializan los estados probabilísticos globales, se invoca la rutina de entrenamiento y se extraen los resultados para analizar cuantitativamente cómo el modelo interpreta la mezcla de densidades.

Interpretación Matemática de los Componentes del Script:

1. **Fijación de la Semilla Numérica (mx->random->seed)**: Los procesos probabilísticos dependen críticamente de los puntos flotantes de partida. Al definir una semilla fija (2), garantizamos que cada ejecución del cuaderno en las computadoras de los estudiantes arroje exactamente los mismos centros iniciales y el mismo número de iteraciones hacia el óptimo local.
2. **Extracción de Métricas de Convergencia**: El algoritmo evalúa el comportamiento histórico de la **Log-Verosimilitud** (\$log_likeliheids). En la fase de diagnóstico estadístico, los estudiantes deben verificar que este indicador dibuje una curva estrictamente ascendente, estabilizándose de forma asintótica a medida que el error marginal decrece por debajo del límite de tolerancia (1e-4).
3. **Del Clustering Difuso al Clustering Duro (argmax)**: Aunque GMM calcula la probabilidad continua de que cada muestra pertenezca a los K componentes gaussianos (almacenada en la

matriz `$responsibilities` de dimensiones $[N, K]$), a menudo se requiere tomar una decisión final de clasificación. El operador matemático `argmax` evalúa horizontalmente las filas del tensor:

$$z_n = \arg \max_k \gamma_{nk}$$

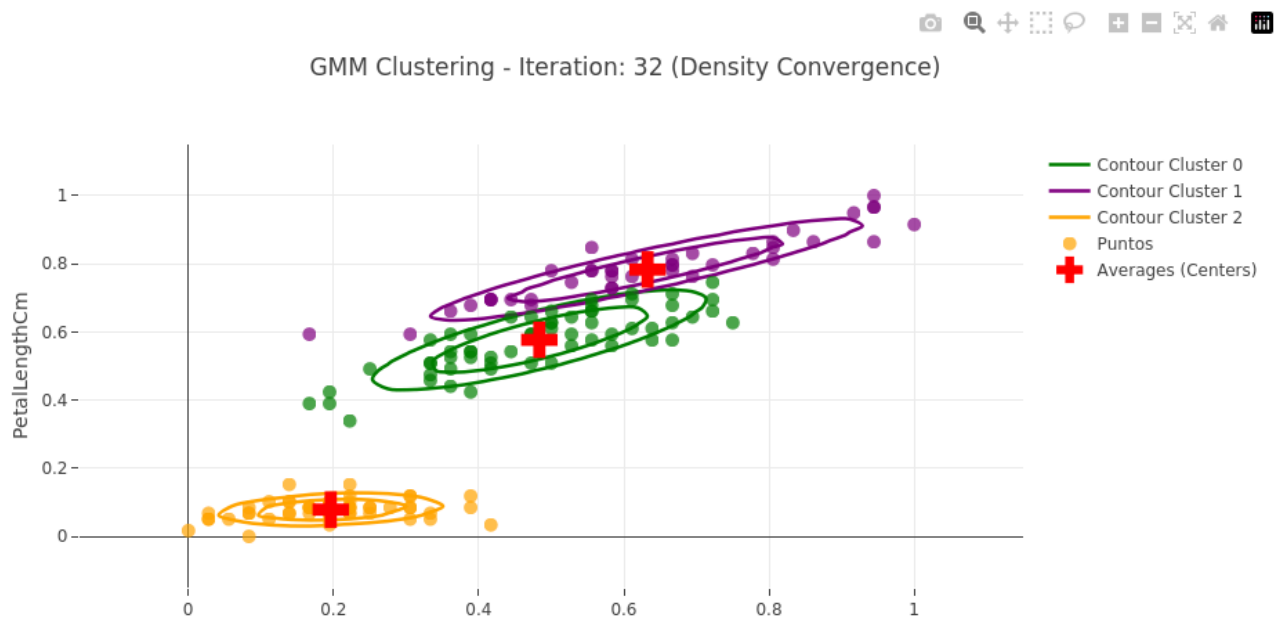
Esto extrae el índice del componente con mayor confianza probabilística, transformando las responsabilidades blandas en etiquetas discretas definitivas (`$assignments`) listas para ser visualizadas.

[illegible]

Out[10]: 1

```
In [11]: # FINAL CONVERGENCE
my $plot = sml->plot_current_state(
  $X_normalized,
  $gmm_means,
  $gmm_covs,
  $assignments,
  $num_iters,
  $header_2d,
  $K
);
# show_plot($plot);
# IPerl->display($plot);
sml->embed_plot($plot, width=>850, height=>450);
```

```
In [12]: sml->embed_plot($plot, width=>850, height=>450);
```



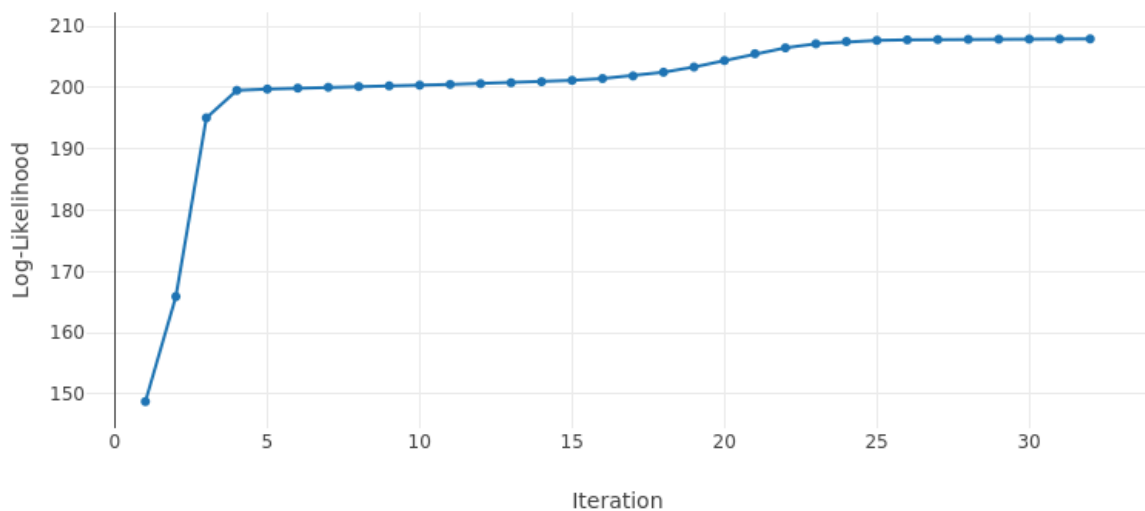
```
In [13]: # Graficar la evolución de la Log-Verosimilitud
my $trace = new Chart::Plotly::Trace::Scatter(
  x => [ 1 .. scalar(@$log_likelihoods) ],
  y => $log_likelihoods,
  mode => 'lines+markers',
  name => 'Log-Likelihood'
);

my $layout = {
  title => { text => 'Evolution of Log-Likelihood in EM (GMM)' },
  xaxis => { title => 'Iteration' },
  yaxis => { title => 'Log-Likelihood' },
  width => 850, height => 450
};

my $plot = new Chart::Plotly::Plot(traces => [$trace], layout => $layout);
# show_plot($plot);
# IPerl->display($plot);
# sml->embed_plot($plot, width=>850, height=>450);
```

```
In [14]: sml->embed_plot($plot, width=>850, height=>450);
```

Evolution of Log-Likelihood in EM (GMM)



```
In [15]: sub gen_data {
  my ($self, %args) = @_;
  my $K                = $args{k}                // 3;
  my $dim               = $args{dim}              // 2;
  my $points_per_cluster = $args{points_per_cluster} // 200;
  my $lim               = $args{lim}              // [-10, 10];

  my @all_points;

  # 1. Generar K medias aleatorias dentro del rango [lim[0], lim[1]]
  # Formula: rand * (max - min) + min
  my $range = $lim->[1] - $lim->[0];
  my $means = nd->random->uniform(low => 0.0, high => 1.0, shape => [$K, $dim]) * $range;

  for my $i (0 .. $K - 1) {
    my $mean_i = $means->slice($i);

    # Generar una covarianza aleatoria pero compacta para que no se encimen los clústers
    my $cov_rand = nd->random->uniform(low => 0.1, high => 0.5, shape => [$dim, $dim]);

    # Controla el radio/dispersión de la nube.
    # Si aumentas de 0.5 a 1.2, la nube se vuelve gigante y más dispersa.
    my $dispersion = 0.5;
    my $cov_i = nd->eye($dim) * $dispersion;

    my $samples = multivariate_normal->rvs($mean_i, $cov_i, $points_per_cluster);
    push @all_points, $samples;
  }

  # 4. Concatenar verticalmente todos los componentes en un único tensor de datos X
  return nd->concat(@all_points, dim => 0);
}
sml->add_to_class('gen_data', \%gen_data);
```

Out[15]: *sml::gen_data

```
In [16]: sub plot_generated_data {
  my ($self, $X, %args) = @_;
  my $dim = $X->shape->[1];

  # El código original en Python solo grafica si la dimensión es estrictamente 2D
  if ($dim != 2) {
    print "Automated plotting is only supported for two-dimensional environments (dim: $dim)";
    return;
  }
}
```

```

}

# Extraer columnas cartesianas X e Y
my $x_coords = $X->slice(':', 0)->asarray;
my $y_coords = $X->slice(':', 1)->asarray;

my $scatter_trace = new Chart::Plotly::Trace::Scatter(
    x      => $x_coords,
    y      => $y_coords,
    mode   => 'markers',
    marker => {
        size      => 4,
        opacity   => 0.5,
        color     => 'blue'
    },
    name    => 'GMM Synthetic Data'
);

my $layout = {
    title => { text => 'Synthetic Dataset (Gaussian Mixtures)' },
    xaxis => { title => 'Feature 0', przerw => 1 },
    yaxis => { title => 'Feature 1', przerw => 1 },
    width => 850,
    height => 500
};

return new Chart::Plotly::Plot(traces => [$scatter_trace], layout => $layout);
}
sml->add_to_class('plot_generated_data', \&plot_generated_data);

```

Out[16]: *sml::plot_generated_data

```

In [17]: # Generar la estructura de datos puramente matemática
mx->random->seed(42);
my $X_synthetic = sml->gen_data(
    k           => 3,
    dim         => 2,
    points_per_cluster => 150,
    lim         => [-10, 10]
);

printf "Successfully generated synthetic matrix. Dimensions: %s\n",
    dump($X_synthetic->shape);

```

Successfully generated synthetic matrix. Dimensions: [450, 2]

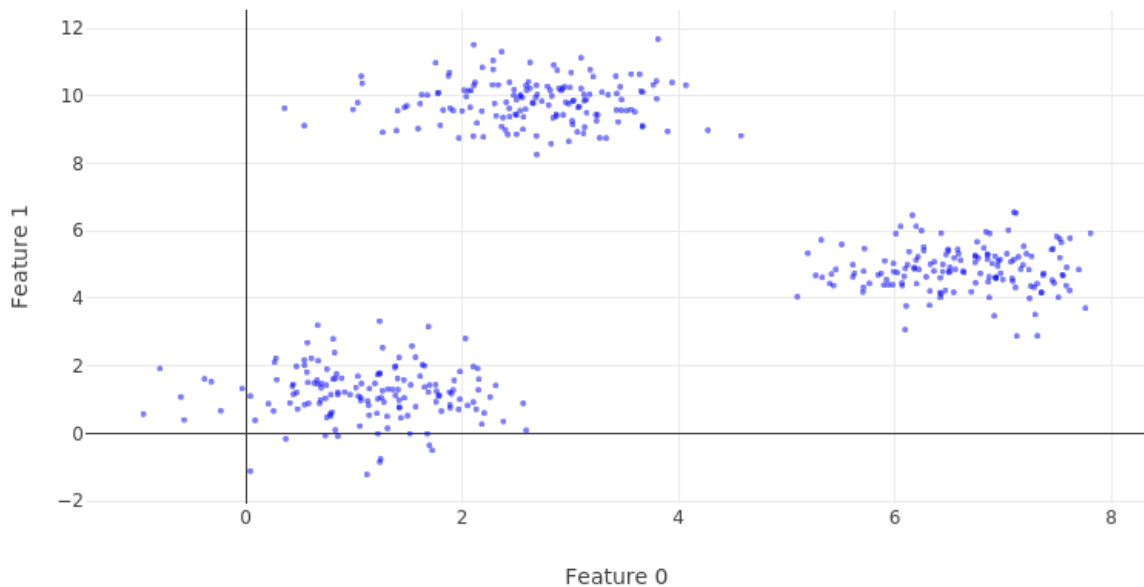
Out[17]: 1

```

In [18]: # Renderizar de forma aislada e independiente
$plot = sml->plot_generated_data($X_synthetic);
# show_plot($plot);
# IPerl->display($plot);
sml->embed_plot($plot, width=>850, height=>500);

```

Synthetic Dataset (Gaussian Mixtures)



```
In [19]: mx->random->seed(3);
my $K = 3;

# Ajuste iterativo del Modelo de Mezclas Gaussianas
my ($responsibilities, $gmm_means, $gmm_covs, $log_likelihoods, $plots) =
  sml->fit($X_synthetic, $K,
    max_iter    => 50,          # <--- PERMITIR ITERACIONES
    tol         => 1e-4,       # <--- TOLERANCIA ESTÁNDAR
    header      => $header_2d,
    plot_steps  => 0,
    init_params => 'random_from_data'); # 'random_from_data'
```

Convergence achieved in the iteration 4.

```
Out[19]: <AI::MXNet::NDArray 450x3 @cpu(0)><AI::MXNet::NDArray 3x2 @cpu(0)>ARRAY(0x55b7467c58d0)ARRAY(0x55b746715e70)ARRAY(0x55b7467162d8)
```

```
In [20]: # =====
# FINAL RESULTS
# =====

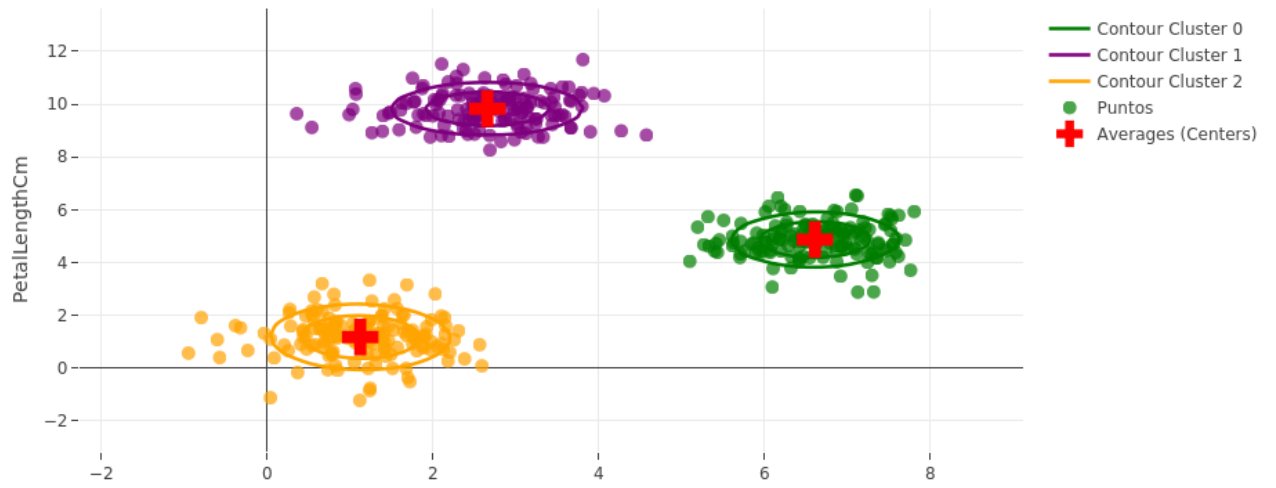
print "\n" . "=" x 50 . "\n";
print "          FINAL PARAMETERS\n";
print "=" x 50 . "\n";

# Imprimir Medias Finales
print "Final Means (Centers):\n";
print $gmm_means->asstr, "\n";

# Imprimir Covarianzas Finales
print "Final Covariances:\n";
if (defined $gmm_covs) {
  for my $k (0 .. $K - 1) {
    printf "Component %d Covariance Matrix:\n", $k;
    print $gmm_covs->[$k]->asstr, "\n";
  }
}

# Imprimir Métricas de Convergencia
my $num_iters = scalar(@$log_likelihoods);
printf "Number of iterations to converge: %d\n", $num_iters;
printf "Final Log-Likelihood (Lower Bound Perl): %.6f\n", $log_likelihoods->[-1] if @
# Final assignments
my $assignments = nd->argmax($responsibilities, axis => 1);
```


GMM Clustering - Iteration: 4 (Density Convergence)



In []: